VSCode-Updater Architecture & Design

VSCode-Updater is built around a deterministic, operator-grade update and version-management pipeline for Visual Studio Code on Windows. The design prioritizes reliability, predictable failure modes, hardened silent-install behavior, symlink-based version switching, and audit-transparent logging. The module now exposes a full suite of public commands for updating, rollback, safe-mode execution, version discovery, and ZIP-based fallback installation.

The design prioritizes reliability, predictable failure modes, hardened silent-install behavior, and audit-transparent logging.

This document describes the internal architecture, lifecycle, watchdog model, exit-code normalization, and security boundaries.

The module now uses a consolidated .psm1 design. All public functions are implemented directly in the module file, with helper functions under Private/.

---

1. Design Principles

VSCode-Updater follows these core principles:

- **Deterministic Execution**
  No ambiguous states, no nondeterministic timing, no hidden behavior.
- **Operator-Grade Reliability**
  Clear lifecycle, hardened error handling, predictable exit codes.
- **Minimal Surface Area**
  No external dependencies, no unnecessary configuration, no GUI.
- **Audit Transparency**
  Logs reflect every major decision and lifecycle event.
- **Security by Construction**
  No sensitive data in logs, no unvalidated input, no unsafe process control.

---

2. High-Level Architecture

Update-VSCode │ ├── Resolve-DownloadChannel │ └── Determines stable/insider channel URL │ ├── Download-Installer │ └── Downloads VS Code installer to C:\Logs\Temp │ ├── Invoke-Installer │ ├── Starts installer silently │ ├── Launches watchdog │ └── Waits for completion │ ├── Watchdog │ ├── Monitors installer process │ ├── Detects stalls/timeouts │ └── Terminates hung installers │ └── Normalize-ExitCode └── Maps installer exit code → deterministic module exit state

Code

All components are implemented as internal helpers under `/Private`.

2.1 Multi-Version Architecture

VSCode-Updater now supports multiple installed versions of Visual Studio Code, including:

- System installers
- User installers
- ZIP/portable builds
- Cached fallback builds

A managed symlink (`vscode.exe`) is used to control the active version.
All version switching, rollback, and fallback operations modify only the symlink target — never the underlying installations.

Components

- **Version Discovery Layer**
  Scans known installation paths and ZIP caches to enumerate available versions.
- **Symlink Controller**
  Creates, validates, and updates the active VS Code symlink.
- **Rollback Metadata**
  Tracks previous symlink targets to enable deterministic rollback.
- **ZIP Fallback Extractor**
  Extracts portable builds when the EXE installer fails or is unavailable.

These components operate independently of the update pipeline but share the same logging and deterministic behavior guarantees.

2.2 Public Command Surface

The module now exposes the following operator-grade commands:

- `Update-VSCode` — Primary update pipeline with watchdog monitoring.
- `Get-VSCodeVersions` — Enumerates all detected VS Code installations.
- `Switch-VSCodeVersion` — Updates the symlink to activate a specific version.
- `Invoke-VSCodeRollback` — Reverts to the previously active version.
- `Test-VSCodeSymlink` — Validates symlink integrity and target correctness.
- `Start-VSCodeSafeMode` — Launches VS Code with extensions disabled.
- `Get-VSCodeDashboard` — Displays a structured diagnostic dashboard.
- `Invoke-ZipFallback` — ZIP-based fallback installer for failed EXE updates.

All commands share the same logging model, deterministic behavior, and security boundaries.

---

3. Update Lifecycle

The update process follows a strict, linear lifecycle:

1. **Start Banner Logged**
   (If multi-version mode is active, version discovery and symlink validation occur before download resolution.)
2. **Download Channel Resolution**
3. **Installer Download**
4. **Silent Installer Execution**
5. **Watchdog Monitoring**
6. **Exit-Code Normalization**
7. **Stop Banner Logged**
8. **Return Deterministic Exit State**

Each step logs a timestamped event to:

C:\Logs\Update-Code.log

Code

---

4. Installer Orchestration

The installer is executed with:

VSCodeUserSetup-x64.exe /VERYSILENT /NORESTART

Code

Rules:

- No GUI is shown.
- No user interaction is required.
- Installer output is not captured directly (only exit code).
- The process handle is passed to the watchdog.

Failure Modes

The installer may:

- exit normally
- exit with a known error code
- hang indefinitely
- spawn child processes
- fail silently

The watchdog ensures deterministic handling of all cases.

4.1 ZIP Fallback Installer

When the EXE installer fails, is corrupted, or cannot be launched, VSCode-Updater automatically falls back to a ZIP-based installation path.

ZIP fallback guarantees:

- deterministic extraction
- no GUI
- no elevation required
- full compatibility with symlink switching
- identical logging and watchdog behavior

The fallback is invoked automatically or manually via `Invoke-ZipFallback`.

---

5. Watchdog Architecture

The watchdog is a lightweight monitoring loop that:

- observes installer CPU time
- tracks elapsed wall-clock time
- detects stalls
- enforces a hard timeout
- terminates the installer if required

Watchdog Flow

Start watchdog │ ├── Monitor process every N seconds │ ├── If CPU time unchanged for stall threshold → terminate │ ├── If elapsed time > timeout → terminate │ └── Log watchdog event and return state

Code

Watchdog Guarantees

- Installer cannot hang indefinitely
- Termination is logged
- No sensitive system details are logged
- Behavior is identical across PS5.1 and PS7+

Watchdog Output Format

The watchdog produces three categories of log entries:

- **Activity logs** — printed when progress is detected
- **Transition logs** — printed when the installer changes state (Idle → Active, etc.)
- **Stall logs** — printed when the installer becomes unresponsive

Each stall type maps to a deterministic exit code returned by Update-VSCode.

6. Exit-Code Normalization

Installer exit codes are mapped to deterministic module exit states.

Example Mapping

Installer Code	Meaning	Normalized State
0	Success	Success
1638	Already installed	Success
1603	Fatal error	Failure
Timeout	Watchdog termination	Failure
Stall	Watchdog termination	Failure

Why Normalize?

- VS Code installer exit codes are inconsistent across versions
- Automation systems require predictable states
- Monitoring tools (Nagios, RMM, CI/CD) expect single-line output

---

7. Logging Model

Logs are written to:

C:\Logs\Update-Code.log

Code

Rules

- Append-only
- Hardcoded path
- Filename derived from module basename
- Directory created if missing
- No override parameter
- No sensitive information may appear in logs

Logged Events

- lifecycle banners
- download start/stop
- installer launch

- installer exit code
- watchdog events
- normalized exit state

---

8. Security Boundaries

VSCode-Updater enforces strict boundaries:

- No external dependencies
- No remote execution beyond the installer
- No user-supplied paths or commands
- No environment variable expansion in logs
- No sensitive data logged
- Watchdog does not expose PIDs or system internals

Security posture aligns with `SECURITY.md`.

---

9. Cross-Platform Behavior

VSCode-Updater is designed for:

- Windows PowerShell 5.1
- PowerShell 7+

Behavior is identical across both runtimes:

- same logging
- same exit codes
- same watchdog behavior
- same installer orchestration

---

10. Future Enhancements (Planned)

- Optional log rotation
- Optional `-LogPath` override

- Extended exit-code mapping
- Additional watchdog heuristics
- Telemetry-free diagnostics mode
- Extended multi-version orchestration (Insiders + Stable coexistence)
- Optional version pinning
- Enhanced dashboard metrics

These enhancements will not break deterministic behavior.

---

11. Summary

VSCode-Updater is engineered for:

- deterministic execution
- hardened silent installs
- predictable failure modes
- audit-transparent logging
- minimal surface area
- operator-grade reliability

VSCode-Updater is engineered for deterministic execution, hardened silent installs, predictable failure modes, audit-transparent logging, and multi-version reliability. The addition of symlink-based switching, rollback, safe-mode execution, and ZIP fallback ensures stable, repeatable behavior across all Windows environments and installation types.